CS-GY 6934 Deep Learning Detailed Midterm Report

Thomas Kong
NYU Tandon School of Engineering
CS-GY 6943
tk2558@nyu.edu

Abstract

This paper presents a system for generating valid Scalable Vector Graphics (SVG) code from natural language prompts as part of a midterm project involving a constrained Kaggle competition. The task requires producing syntactically correct, renderable SVG outputs while adhering to strict structural and length constraints. In this paper I report on how I fine-tuned a small instruction-tuned language model using parameter-efficient techniques (LoRA) within the Unsloth framework. Key focuses of approaches are a multi-stage preprocessing pipeline, response-only loss and structured prompt formatting. Experimental results show that preprocessing and data quality have a significantly larger impact on performance than hyperparameter tuning.

1 Introduction

1.1 Problem Statement

Generating structured code from natural language is a challenging task that requires both semantic understanding and strict adherence to syntax rules. In this project, we address the problem of generating valid Scalable Vector Graphics (SVG) code from textual descriptions, as posed in a Kaggle code competition. The task requires producing syntactically valid XML-based SVG outputs that comply with strict constraints, including a fixed canvas size, limited number of elements, and maximum character length.

1.2 Motivation

The primary motivation for this work is to explore how well small language models can be adapted for structured generation tasks in constrained environments. Unlike free-form text generation, SVG generation requires precise formatting, correct nesting, and termination of tags. Failure to meet these requirements results in invalid outputs and zero scores in the evaluation pipeline.

1.3 Summary of Approach

We adopt a parameter-efficient fine-tuning approach using Low-Rank Adaptation (LoRA) on a small instruction-tuned language model. Our method focuses on improving structural correctness through data set cleaning, normalization, and training-time constraints, while maintaining efficient inference within computation limits. Additionally, we incorporate decoding strategies and post-processing steps to ensure valid SVG output.

2 Dataset

2.1 Description of Dataset

For this project, external dataset beyond the provided dataset were prohibited. The dataset we were provided consists of 50,000 rows with three columns: Id, Prompt, and SVG. Each prompt describes a simple icon, image or features, while the SVG represents the rendering.

It is important to note that the provided dataset has a few problems that need to be addressed before we can allow our model to train on it. Not all the training data follows the competition constraints, including a fixed canvas size (256×256), a restricted set of SVG elements, and limits on SVG complexity (e.g., maximum length and path count). Since many of the SVG are not set to a 256x256 canvas, we will need to adjust for this and also scale them to properly fit into one. Otherwise, SVG drawn on a small canvas will appear even smaller on a 256x256 canvas.

Furthermore, going through the data shows that some SVG outputs don't match their outputs. For example, there are many prompts that state something along the lines of "The image consists of a single solid color, which is black, filling the entire frame" (prompt id="aec8cf441d1d214b9d19f5fbaac6ebfc"). However, when you observe the SVG code that these prompts are paired with, they often don't

match this description. In this case, prompt id "aec8cf441d1d214b9d19f5fbaac6ebfc" displays an icon of a battery in SVG.

Finally, some prompts possess garbage characters that were caused by improper encoding. This can often be seen with prompts involving characters with foreign languages such as this prompt: "The image shows the Chinese character 'ÃÂŒÃÂ¸Ãâ¬' (yÃâÃÂ«) in black and 'ÃÂ¦ÃâÃÂ¹' (fÃâÃÂng) in blue, both centered on a white background."

With all this in mind, it was necessary to clean and preprocess our data through a pipeline before we can start training our model on it.

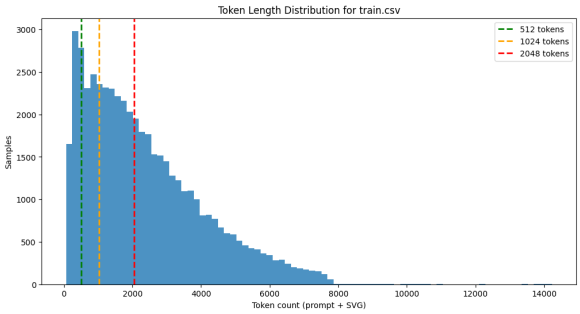

Figure 1: Graph of Token Distribution of all Samples in train.csv

2.2 Pre-processing Part 1

The preprocessing of the dataset was divided into two parts. The first part focuses on a "Data Compression Pipeline" for the SVG data. The primary goal is to optimize and streamline the ‘train.csv‘ dataset into a more efficient train_compressed.csv for model training. This involves two main steps:

1. **Compressing SVGs:** Reducing the length and token count of SVG files to make them more manageable for the model.

2. **Scaling SVGs:** Ensuring all SVGs are scaled to fill a 256x256 canvas, providing a consistent input size for the model.

The pipeline progresses through several stages, involving both external tools (SVGO) and custom Python functions, to clean, transform, and compress the SVG data. First I created a configuration file called svgo.config.transform.js for SVGO (SVG Optimizer), a Node.js-based tool. It defines a set of transformations to be applied to SVGs, such as converting shapes to paths, handling numeric values, and sorting attributes. These transformations standardize and simplify the SVG structure, making them more amenable to further processing. A second SVGO configuration file (svgo.config.compress.js) is also developed for further compression. A dedicated directory structure, /svg_pipeline, is added that include the following folders: svg_in, svg_mid, svg_out, and svg_final. These directories serve as staging areas to organize SVGs at different processing stages of the pipeline

I added several utility and path manipulation functions to assist throughout the pipeline process:

- **basic_svg_cleanup:** Performs an initial, basic cleanup of SVG strings. It removes XML headers, comments, metadata, normalizes whitespace, and removes redundant spacing around equals signs, preparing the SVG for subsequent steps.

- **count_svg_tokens:** Calculates the token length of an SVG string. This is crucial for filtering SVGs later based on a maximum token limit.

- **_parse_translate:** Extracts values from SVG ‘translate‘ transform strings.

- **bake_path_translate:** Modifies SVG paths by directly applying ‘translate‘ transformations to their ‘d‘ (path data) attribute. This helps in removing redundant transform attributes from path elements or groups, simplifying the SVG structure

- **normalize_paths:** Adjusts path coordinates to ensure that the SVG content starts at the (0,0) origin. This normalization step is important before scaling to ensure consistent positioning.

- **scale_svg_to_256:** Scales an SVG to ensure its largest dimension fits and is set within a 256x256 pixel canvas. It adjusts viewBox, width, height, transform coordinates, and all path data (‘d‘ attributes) to reflect the new scale while carefully preserving critical path information like arc flags.

- **clean_degenerate_segments:** Removes zero-length segments from paths, which can be introduced during scaling or transformations.

- **round_path_numbers:** Rounds floating point numbers in path data to a specified precisions, set to two.

- **remove_redundant_transforms:** Identifies and removes transform attributes that represent zero-translations, simplifying the SVG code.

The pipeline flow begins with initial cleanup and staging as the pipeline iterates through the original train.csv DataFrame. Each SVG undergoes ‘basic_svg_cleanup‘ and is saved as an individual ‘.svg‘ file into the ‘IN_DIR‘ if it passes basic validity checks. Then the first SVGO command-line tool is executed, processing SVGs from ‘IN_DIR‘ using the ‘svgo.config.transform.js‘ configuration. These transformed SVGs are outputted to ‘MID_DIR‘ and for each SVG in ‘MID_DIR‘, the helper Python functions are applied to them. SVGs that successfully complete this stage are saved to ‘OUT_DIR‘. A final ‘SVGO‘ run processes SVGs from ‘OUT_DIR‘ using the svgo.config.compress.js configuration. These fully optimized SVGs are saved to ‘FINAL_DIR‘.

For a final filtering and outputting, all successfully processed SVGs from ‘FINAL_DIR‘ are read into ‘optimized_map‘ where a basic validation is performed to ensure proper SVG tag structure. The map is filtered based on token count so only SVGs within the set max SVG tokens are retained to ensure that the generated data is suitable for model input. A new pandas DataFrame, ‘compressed_df‘, is created from the filtered SVGs and it’s contents are saved as train_compressed.csv. This CSV file contains the original prompts paired with their corresponding optimized and token-limited SVG data, ready for subsequent data preprocessing or direct model training.

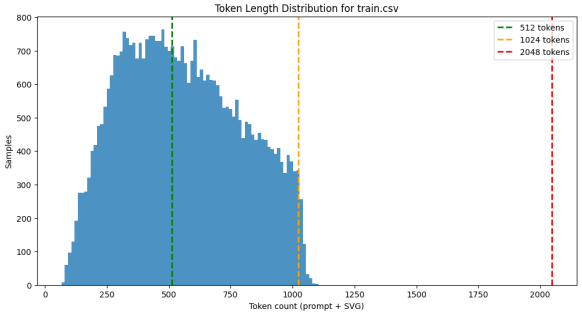

Figure 2: Graph of Token Distribution of all Samples in train_compressed.csv

2.3 Pre-processing Part 2

Part two of Preprocessing builds upon the compressed data (train_compressed.csv) generated in Part 1. The objective is to further refine the dataset by identifying and correcting issues such as incorrect SVG outputs, encoding problems in prompts, and other data anomalies. This stage involves loading the compressed data and applying several custom cleaning functions to both prompts and SVGs. This rigorous cleaning ensures that the model is trained on high-quality, consistent data.

For the purposes of data cleaning, different functions were defined to validate and clean both SVG content:

- **_pick_first_non_empty:** Helper function to safely retrieve a non-empty value from a list of potential fields in an example.
- **clean_svg:** Second round of SVG cleanup and validation.
- **load_source_dataset:** Loads the training_compressed.csv specified. It can handle both HuggingFace datasets and local CSV files. At the end, it filters out examples where either the prompt or SVG is empty after cleaning, reporting the number of usable rows

However, we also have to validate and clean data based on the prompts we were given. As mentioned before, there are prompts with incorrect SVG associated with them and prompts with garbage characters. As a result, I defined functions and helper functions that would examine the prompts in the training data:

- **is_solid_color_prompt** Detects if a prompt describes a single solid-color image by checking for specific keywords and phrases (e.g., "single solid," "entire visible area").
- **extract_color:** Extracts the color name from a prompt if a solid color is detected, defaulting to "black" if no specific color is found.
- **generate_solid_svg:** Generates a standardized SVG string for a solid-colored rectangle filling the entire 256x256 viewBox based on the extracted color.
- **clean_training_prompt:** Cleans the input prompts by fixing common encoding issues (e.g., ‘Ãƒâ€Â,Ã‚Â‘) using ‘ftfy‘ Python Library, removes meta-instructions (e.g., "don’t use markdown") and cleans up leftover punctuation.

- **should_keep_training_prompt:** Determines if a cleaned prompt is suitable for training. It rejects prompts that are too short, still contain encoding artifacts, or are solely meta-instructions.

- **to_prompt_svg:** Applies prompt cleaning fucntions to a prompt. If the cleaning results in an invalid or empty prompt/SVG, it returns empty strings to filter out bad data.

By systematically addressing common data quality issues in both the prompts and SVGs, including correcting known patterns like solid-color image prompts, and filtering out problematic entries, this part of preprocessing ensures that the downstream model training process receives high-quality, consistent, and relevant data.

In summary, preprocessing plays a vital role in our pipeline and the following steps were applied to the dataset:

- **SVG Optimization:** Used SVGO to remove unnecessary metadata, comments, and redundant attributes.

- **Normalization:** Standardized viewBox and dimensions to ensure consistency across samples.

- **Token Cleaning:** Removed malformed or invalid SVGs that could harm training.

- **SVG Correction:** Ensured all SVGs follow a consistent structure and ordering of attributes.

These steps were designed to reduce noise in the dataset and improved the model's ability to learn valid SVG syntax. Once the dataset is loaded and completely cleaned, it is shuffled before being split into a training and evaluation set. This split is crucial for training the model and then assessing its performance on unseen data. Finally both the dataset are mapped to be transformed and formatted into the specific chat-like template (chatml) required for SFT training with models like Qwen:

By the end, we have transformed the cleaned raw data into a structured format suitable for fine-tuning a model through consolidating data, creating distinct training and evaluation sets, applying a consistent SFT chat template, and performing detailed metric analysis. Training examples that result in an empty text field due to empty prompts or SVGs after previous cleaning are filtered out. This ensures that the model training in subsequent steps will proceed with high-quality, appropriately formatted input.

```
<|im_start|>system
{SYSTEM_PROMPT}<|im_end|>
<|im_start|>user
{example['prompt']}<|im_end|>
<|im_start|>assistant
{svg}<|im_end|>
```

Table 1: SFT Input Text Format

Training Set	**Max**	**Average**
Characters length	2720	1145.79
Tokens	1191	635.49
Path Count	28	1.97
Evaluation Set	**Max**	**Average**
Characters length	2228	1140.56
Tokens	1170	628.66
Path Count	16	1.98

Table 2: Metrics for Training and Evaluation Set

3 Model Description

3.1 Architecture

We use Qwen2.5-3B-Instruct, a transformer-based auto regressive language model. The model (unsloth/Qwen2.5-Coder-3B-Instruct-bnb-4bit) is designed for instruction-following tasks and is capable of generating structured outputs such as code. We fine-tune the model using parameter-efficient techniques via the Unsloth framework to reduce memory usage and training time. Specifically, low-rank adaptation (LoRA) is applied to selected attention and feed-forward layers, allowing us to update a small subset of parameters while keeping the majority of the pretrained weights frozen. This significantly reduces GPU memory usage and training time while maintaining strong performance.

Given the structured nature of SVG code, the model must learn both natural language understanding and strict syntax generation. Unlike free-form text generation, SVG outputs require precise token sequences, including nested tags, attribute formatting, and coordinate consistency. So it is important to prioritize the model learning formal structure in addition to semantics.

4-bit quantization was used during training,

which further reduces memory requirements and enables fine-tuning on limited hardware. Despite the reduced precision, the model retains sufficient capacity to learn the mapping between prompts and SVG outputs.

Tokenization is performed using the model's native tokenizer. However, SVG code introduces many special symbols (e.g., '<', '>', '/', '='), which are fragmented into subword tokens. This increases sequence length and can make learning syntax more difficult. To mitigate this issue, consistent formatting and preprocessing was used to reduce variability in token sequences.

3.2 Approach

The task is framed as a sequence-to-sequence generation problem:

- Input: natural language prompt
- Output: SVG string

With this in mind, the model must be trained to generate valid SVG outputs with specific constants. My approach focused on the training stability on structured generation through prepocessing and System Prompt formatting. Since small syntax errors can invalidate an entire SVG, the model must also achieve high token-level accuracy.

Overall, my approach combines efficient fine-tuning of a pretrained language model with careful data curation and decoding constraints to address the challenges of structured SVG generation.

4 Experimentation

4.1 Hyperparameter Settings

Generally, hyperparameter were set to keep the model training for around 6 to 8 hours. However, the time it takes to finish training model also depends on other factors like the notebook environment and GPU currently being used to run the model. The key hyper-parameters that I used during training were:

- Model Name: unsloth/Qwen2.5-Coder-3B-Instruct-bnb-4bit
- LoRA Rank: 16,
- LoRA Alpha: 64,
- Learning rate: 2e-4
- Epochs: 1
- Max sequence length: 2048

The parameters lora_r and lora_alpha are for Low-Rank Adaptation (LoRA) with lora_r referring to the rank of the update matrices and lora_alpha is a scaling factor. The choice of setting lora_r=16 ensures parameter efficiency and reasonable learning capacity, while lora_alpha=64 provides a significant scaling boost (4x) to ensure that the LoRA adaptations effectively influence the model's output to master the SVG generation task.

Learning rate is the step size at which the model's weights are updated during training.For fine-tuning a pre-trained model, a smaller learning rates (e.g., 2e-4) was used as the model already has strong foundational knowledge and isn't being trained from scratch.

Epochs defines the number of full passes over the entire training dataset. Given how large the dataset is, I set epoch to one in order to limit the total training time the model will undergo.

Max sequence length defines the maximum number of tokens that the model can process in a single input sequence. It sets the 'context window' for the model. For this text-to-SVG task, it determines how long the input prompt can be and how long the generated SVG output can be. Although the maximum sequence length is set to 2048, this was set as a precaution as all the training data and max generation output was set to have a maximum of 1024 tokens. If the maximum sequence length is too long, Important information in longer prompts might be truncated, and the model might not be able to generate complete or complex SVG code. On the other hand, setting max sequence length too high will increase memory consumption and computational time.

4.2 Training Details

The model was trained using the Unsloth framework for efficient fine-tuning. Mixed precision training and quantization were used to reduce memory usage. Training was performed on GPU with careful monitoring of training and validation loss.

Some important details when training the model with SFT was that packing was set to False. Packing refers to an optimization technique where multiple short input sequences are concatenated (packed) into a single, longer sequence to efficiently fill the maximum sequence length of the model. This can be beneficial because it reduces padding tokens to make more efficient use of GPU memory and

computation. It can also lead to faster training by processing more actual content per batch. However, by setting it to False during training each entry in the batch corresponds directly to one example from of the dataset. This reinforces the model to learn the SVG format structures and it's behavior when generating output.

Assistant_only_loss was also set to True to mask the System Prompt format and user format for the model. This is a crucial setting for instruction-tuned models as it ensures that the model only calculates and optimizes the loss for the tokens generated by the 'assistant'. In this case, the SVG code is generated by the 'assistant' so the model effectively ignores the system prompt and user query during loss calculation. This helps the model focus on learning to generate correct responses rather than repeating the input.

Finally, the model was set to train on responses only as well during the training. The Unsloth helper function train_on_responses_only further refines the loss masking. It sets specific instruction_part and response_part tokens (<|im_start|>system and <|im_start|>assistant respectively). This explicitly tells the trainer where the assistant's response begins, so the loss is only computed on the assistant's output, preventing the model from being penalized for predicting the instruction or user prompt

4.3 Methodology (What I Tried)

I fine-tune a pretrained language model (Qwen2.5-Coder-3B-Instruct-bnb-4bit) using the Unsloth framework on prompt-SVG pairs. I chose this particular model because it is a lightweight, high-performance language model specialized for coding tasks. It is a 4-bit quantized version of the Qwen2.5-Coder-3B-Instruct model, optimized using the BitsAndBytes (bnb) library to reduce memory usage while maintaining strong reasoning capabilities. This model possess 3.09 Billion parameters and is specifically instruction-tuned for code generation, code reasoning, and code fixing which is relevant to SVG code generation. Originally I had used a smaller model (Qwen2.5-Coder-1.5B-Instruct-bnb-4bit) but when the project was updated to allow models sizes of less than 4B, I upgraded to Qwen2.5-Coder-3B-Instruct-bnb-4bit.

I did experiment with other models including a Vision Model (unsloth/Qwen3-VL-2B-Instruct-unsloth-bnb-4bit) in attempt to leverage it's capacity to for vision detection for outputting valid, visual SVGs. Originally, I tried to use the model provided in the starter-code (unsloth/Qwen3.5-2B-Instruct-bnb-4bit) but this model did not exist as I looked through Unsloth's Hugging Face Account. When I was trying to finding unsloth/Qwen3.5-2B-Instruct-bnb-4bit, unsloth/Qwen3-VL-2B-Instruct-unsloth-bnb-4bit was the first result so I attempted to use the vision language model. However, the implementation and training complexity of the Vison Model resulted in it often relying on a fallback SVG during generation. I also tried using a more standard (unsloth/Qwen2.5-3B-Instruct-bnb-4bit) which did have better success. Ultimately chose to use Qwen2.5-Coder-3B-Instruct-bnb-4bit given that it was specifically instruction-tuned to understand code better and found more success with it.

For preprocessing, I developed a data pipeline that focuses heavily on preprocessing SVG data to ensure consistency and reduce noise. It applied SVG optimization, normalization, and filtering to improve training quality. In order to ensure that the pipelining did not distort the new compressed SVG training outputs into something unrecognizable from it's original, I also sampled and reviewed different compressed SVGs and compared them to their original.

As mentioned in Training Details, I focused on improving the model's ability to predict and generate valid SVG structures during it's training. I set packing to False, Assistant_only_loss to True and used train_on_responses_only in order to help my model learn SVG outputs better. Furthermore, when splitting my dataset into training and evaluation I set the evaluation size of the evaluation set to 5% of the dataset size in order to provide unseen data for the model to assessing on during training. During training, I also found out that the warm-up ratio configuration was also deprecated so I switched to warm-up steps for training arguments. Warm-up steps in represent an initial phase where the learning rate gradually increases from a low value to a target maximum to prevent early training instability and optimize convergence. This approach aims to mitigate large gradients causing divergence in initial training and ensure more stable optimization.

During inference, I set the model to evaluation mode and fast inference in order to save time during output generation. I also used torch.inference_mode() to optimize the model for deployment and set do_sample to False for greedy decoding.

Pytorch's torch.inference_mode() disables its

gradient calculation mechanisms. During training, PyTorch builds a computational graph to store information needed for backpropagation (calculating gradients). However, for inference, only the forward pass (generating predictions) matters so gradient calculation is unnecessary. By disabling gradient calculation it frees up this memory and without the overhead of building and storing the computational graph, the forward pass runs faster.

Normally, if do_sample is set to True, the model would sample from the probability distribution of possible next tokens. This is done to introduce randomness and leads to more diverse, creative, but less predictable outputs. However with greedy decoding, do_sample=False, the model predicts the next token by simply selecting the token with the highest probability according to its learned distribution at each step of the generation process. For tasks where a precise, deterministic output is desired (e.g., generating structured data like SVG), greedy decoding ensures that the same prompt always yields the same result. By avoiding the computational overhead of sampling from a multinomial distribution at each step that comes with do_sample being set to True, output generation is generally faster than sampling methods.

I also experimented with several improvements including:

- Various SVG preprocessing strategies
- Adjusting SVGO Configurations
- System Prompt Formatting Variations
- Training on response only

In the beginning, I did basic preprocessing by simply filtering out any training data from the dataset that didn't meet the specific constraints: fixed 256x256 canvas, greater than maximum length and path count, etc. However, this heavily limited the amount available data to train my model on and providing a large, diverse dataset is vital. So I went through the training data and noticed a few things. Particularly that there were some garbage data that needed to be accounted for and that many of the SVG training data had long outputs that could be compressed. As a result, I shifted to creating a data pipeline with SVGO to optimize SVG and prompt pairs.

I also adjusted my System Prompt that was used to feed the model information. Originally it was a simple prompt that stated "You generate compact, valid SVG markup from user requests. Return only SVG code with a single root <svg> element." However, I expanded on this original system prompt by specifying clears rules to the model in order to reinforce these constraints. As a result, my new System Prompt became "You are an SVG code generator. When given a description, output ONLY a single valid SVG with these rules:

1. Fill the full viewBox — shapes should be large and centered, not small or tucked into a corner.
2. Use solid fills and simple strokes. No masks, filters, or external references.
3. End output with </svg>."

In this situation, I traded extra tokens for the System Prompts in order to get my model to learn and obey these rules. I added Unsloth's training on response only helper function later on upon reading it's documentation. At first, I noticed that it increased the training and validation loss on my model during training. However this is due validation only applying to the assistant's output and is no now longer predicting the instruction or user prompt. Most likely, the System Prompt instruction was inflating these scores originally as it would be consistent in every input. So now my model is more effectively learning to predict the SVG training outputs. Overall, preprocessing and training on response only had a larger impact than most hyperparameter changes.

5 Results

5.1 Quantitative Results

Model performance was evaluated using the competition metric, which combines:

- Visual fidelity (SSIM + Edge similarity)
- Structural similarity (tree edit distance)
- Compactness (SVG length)

As seen in Table 3, you can observe steady improvements in the model for validation loss during training, with diminishing returns after early awhile. Although the validation and training loss may seem quite high, this shows the model is training on responses only. It's no longer trying to predict System prompt and prompt which would pad the training data. Instead it's focusing only on learning the SVG training data output.

Step	Training Loss	Validation Loss
100	0.741200	0.738570
200	0.718400	0.719605
300	0.708500	0.704275
400	0.693800	0.695250
500	0.708800	0.685484
600	0.674000	0.678989
700	0.690300	0.673436
800	0.651500	0.666361
900	0.647100	0.660865
1000	0.658200	0.656777
1100	0.681000	0.651031
1200	0.663900	0.646236
1300	0.656800	0.641730
1400	0.649700	0.638821
1500	0.638700	0.636132
1600	0.636200	0.634530
1700	0.645200	0.633608
1800	0.649300	0.633401

Table 3: Model's Training Results

The model was successful in outputting valid SVGs when given a prompt a majority of the time. Of the 1000 test prompts the model was asked to generate for, approximately 10% of the time the model relied on a fallback SVG. In these cases, it can be observed that the model truncated and hit it's max token limit during generation. As a result, it produces an invalid SVG that does not end with </svg> and isn't closed. Although this can be solved by increasing the max token limit during generation, this comes at the cost of higher computation time to output these generations. Unfortunately, the model did not perform as well as hoped or predicted given that it scored in the bottom 50% of submissions in the Kaggle's competition leaderboard with a public score of approximately 11%.

5.2 Comparisons

In the Kaggle's competition leaderboard, the highest score is approximately 18.3% with an average score of 13.7% and median score 13.8%. Unfortunately, my model falls under the average and median score which means that there is room for improvement for my model. In the future, I would like to examine the models of the top ten percent of the leaderboard to see what I could have done differently and better.

Currently I suspect that the biggest differences between my model and theirs are base model choice, preprocessing and prompt analysis. Although, I believed the Qwen2.5-Coder-3B-Instruct-bnb-4bit was the best model to fine tune for this situation it is possible there were better options. Most likely, others used a more updated and newer Qwen 3.5 model to fine tune. Qwen 3.5 is a upgrade over Qwen 2.5, focusing on native multi-modal capabilities and improved reasoning though it does require higher computing resources. I stuck with Qwen 2.5 based on it being denser and attuned to text-only inputs like SVG code.

It is also likely that other competitors had more refined data pipelines for preprocessing than mine. Despite trying to account for any irregularities after compressing and scaling training data, there were a few bugs with training SVGs that involved translation transformations. This caused some SVG to be inaccurate after going through the pipeline. Better prompt analysis was a feature through a keyword extractor I had consider implementing if I had more time. By breaking down prompts into only it's most important parts or tags, I believe that the model would better learn to associate tags with certain parts of the SVG code for better reproducibility during output generation.

5.3 Ablation Studies

Ablations were conducted on:

- With vs Without SVG optimization
- Different Preprocessing pipelines
- Training on Response Only
- Max Token Length: 512 tokens vs 1024 tokens

At first, I freely trained my model on all the data in the dataset without any SVG optimization. The only modification I made to the data was setting all their canvas sizes to a fixed 256x256. The result was that the model could barely generate any valid SVG outputs from the prompts. When examining the prompts the model produced, I observe that it generally built paths with long, redundant numerical values that it learned from the training dataset. In response to this, I began developing a data compression pipeline to round redundant numerical values in the SVG training data which also help save token space for the model to generate my relevant code to complete a valid SVG. This significantly improved the efficiency and effectiveness

of my model while proving that the data the model trains on is highly important to it's success.

Following that, I reviewed the list of outputs my model produced and observed a new common struggle. The model had difficulty generating smaller icons and were not centered properly. I theorized that this was a result of the all the training data being set to a fixed 256x256 canvas. This caused icons that had small canvas sizes to appear even smaller now while also off-setting its boundary borders. As such, I experimented in trying to scale canvas sizes to properly fill the space of a 256x256 canvas. The result was developing a second part to the data pipeline after the initial compression since after scaling, the SVG needs to be compressed again in case it had any long numerical values after the process. This change from using one data preprocessing pipeline to using two data preprocessing pipeline shows an increase in the quality of my outputs after training. Prior to this change, the model would produce small, almost unrecognizable icons. Now the model produced larger, clearer and centered to the page images more consistently.

Before making my model train on response only and masking everything but the assistant part of the input, the model showed lower validation loss results during training. Validation and training loss was around 40% to 30% during this time but it do not appear as though the model actively reflected this for generation. Upon researching the documentation for Unsloth and SFTTrainer, I came across assistant_only_loss and train_on_responses_only. Both functions help to mask user prompts and only calculate loss on the assistant's response to improve fine-tuning efficiency and prevent overfitting on inputs. Unsloth's trainn_on_responses_only modifies the trainer to mask inputs between instruction part and response part, which are part of chat template I was using. While assistant_only_loss is a configuration flag in SFTConfig that directs the SFTTrainer to only compute loss on assistant tokens. Both were used to prevent the model from learning to generate user questions which allowed it to focus strictly on producing high-quality responses. This addition to the model helped to improve the efficiency and effectiveness of it's training to output valid SVGs.

When setting max tokens to 512, output generally suffers as it quickly reaches this limit during generation and cuts off. This results in an incomplete SVG code and outputs the fallback SVG as a default. As such, changing max tokens to a higher value (1024) was necessary especially since the data the model was trained on consisted of SVG codes with 1024 or less tokens.

6 Conclusion

6.1 Summary of Findings

In this work, I explored the problem of generating valid SVG code from natural language prompts under strict structural and evaluation constraints. Our experiments demonstrate that data quality is the most critical factor for success in structured generation tasks. In particular, the introduction of a multi-stage preprocessing pipeline—consisting of SVG optimization, normalization, scaling, and prompt cleaning—significantly improved both output validity and visual quality.

Without preprocessing, the model frequently produced invalid SVGs with redundant numerical values, malformed structures, and poor rendering behavior. In contrast, training on cleaned and standardized data enabled the model to generate more consistent and syntactically correct outputs. Additionally, enforcing response-only training and structured formatting further improved the model's ability to focus on generating valid SVG code.

While hyperparameter tuning and architectural choices (e.g., LoRA configuration, gradient strategies) contributed to training stability, their impact was relatively minor compared to preprocessing improvements. It was also observed that generation constraints, particularly maximum token length, directly affected output completeness, with lower limits leading to truncated and invalid SVGs.

Despite these improvements, the model achieved only moderate performance on the Kaggle leaderboard, indicating that challenges remain in achieving high visual fidelity and structural similarity. Overall, this work highlights the importance of data-centric design and preprocessing in enabling language models to perform well on constrained, structured generation tasks such as SVG synthesis.

6.2 What Worked and What Didn't

Worked:

- SVG normalization and cleaning
- Consistent formatting
- Parameter-efficient fine-tuning

SVG normalization and cleaning assisted in providing comprehensible data for the model to train on. With consistent formatting the model

was able to learn what to do for specific outputs. For example, the model was consistent in adding circles when specified by prompts and the to output a black rectangle that filled up the canvas when given prompts that were similar to "the image consists of a single solid color filling the entire frame". This is likely due to the consistent formatting done to SVG training data for these prompts during preprocessing. High parameters for LoRA Alpha fine-tuning worked to better enforce the model to learn SVG formatting.

Didn't Work Well:

- Training on noisy or inconsistent SVG data
- Low max tokens

When training with noisy or inconsistent SVG data, the model often produced outputs with long values when drawing paths. This was the result of the model learning from uncompressed SVG training data that had long values when drawing paths. This did not help when setting the max tokens to a low value (e.g., 128 or 512) as the generation will typically reach these max limits and truncate.

6.3 Future Directions and Improvements

In the future, one idea I had that I didn't have time to implement was to utilize YAKE! (Yet Another Keyword Extractor) while scrubbing my data and prompts. YAKE is a light-weight, unsupervised keyword extractor that uses statistical features to select the most relevant keywords. My idea for YAKE was to use it to filter out unnecessary words in a prompt in order to better train the model. For example, a prompt in the training data could be: "The image features two orange squares with a microphone icon and an arrow connecting them, set against a white background.". There is a lot of noise in this given training prompt but the most important information that the model needs understand to generate is: "image features two orange squares", "microphone icon", "arrow connecting them", and "white background". By extracting these keywords, the model can ideally learn them as tags to better associate SVG drawings and paths to these tags whenever it needs to generate an output based on a given prompt.

Additionally, if given more time, I would work on my Data Compression Pipeline further as there are some bugs with the pipelines. Certain training data are sometimes affected during the compression and scaling process, resulting in the new compressed SVG not looking exactly the same has the original.

Beyond that, with more time and resources I could train my longer for potentially better results. More data from a diverse dataset and a greater number of epoch during training could help improve my model during output generation. I believe I could have also experimented more with using different System Prompts during generation. For my System Prompt, I defined strict rules for the model to follow but in exchange I used extra tokens. There may have been a better balance between using less tokens for the System Output while also providing clearer rules for the models to consider during generation to ensure it wouldn't output a poor SVG generation. Finally, my choice in using a Qwen2.5 Model may have been shortsighted and I should have tried using a Qwen3.5 Model instead. Originally, I used a Qwen2.5 Model because it had a Coder Instruct bnb 4bit version that I believed would worked well for this project. However, it is possible that the benefits of a newer Qwen3.5 would have overweigh those benefits so given more time, experimentation with newer models should be explored.

Overall, there are a lot of room for growth in my project and improvement. Unfortunately, I was mainly limited by time and resources with Google Colab's and Kaggle's GPUs being limited in use for free users. This resulted in less opportunities for testing and experimentation. However, this model currently does serve as a good basis and foundation for future fine tuning with updates.

7 Acknowledgment

I would like to acknowledge that ChatGPT was consulted for coding and debugging purposes when doing this project. It particularly assisted in the developing preprocessing data compression pipeline.

Links

7.1 GitHub

- GitHub Repo Link with Code

7.2 HuggingFace

- Model Weights Saved in HuggingFace
- Qwen2.5-Coder-3B-Instruct-bnb-4bit
- Public Version of train_compressed Dataset